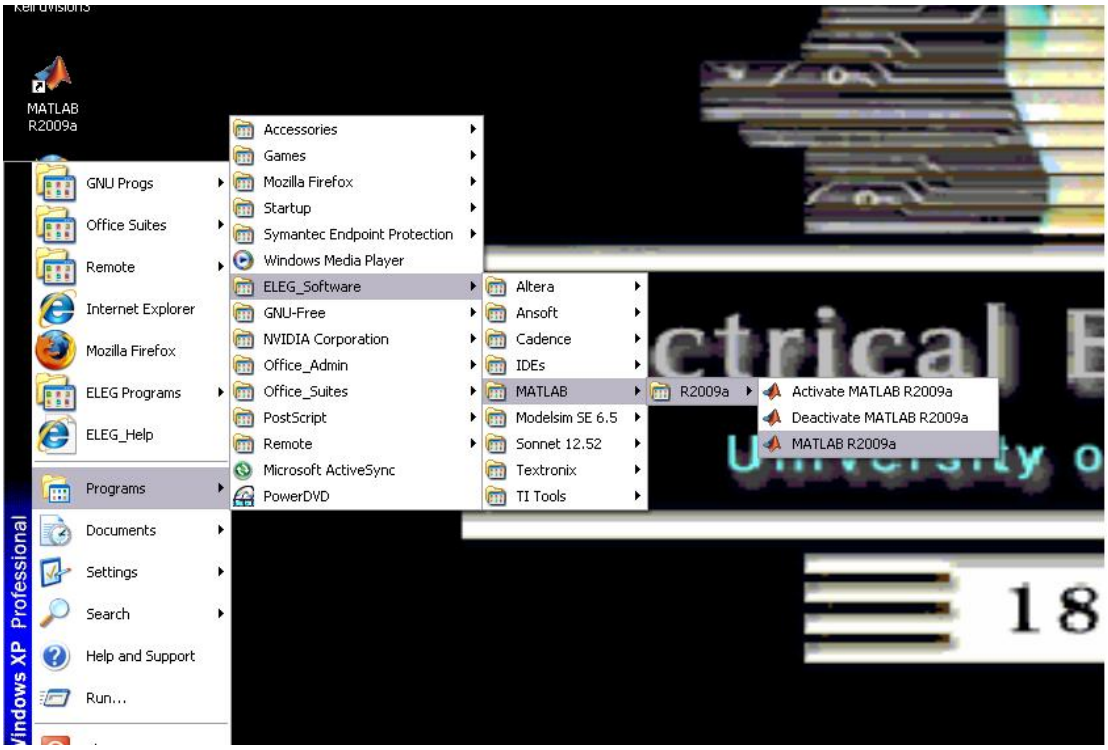
MATLAB Basics

1. Access MATLAB Software in any GACL machine in the ELEG Department.
2. To Launch MATLAB:

Start →Programs →ELEG_Software →MATLAB →R2009a →MATLAB R2009a



3. Use a folder in your Desktop (or preferably Samba K: Network drive) as your "Work Directory". Inside MATLAB software, go to *File →Set Path* then *Add Folder →Browse* to the folder and OK/Save/Close. Any file that is present in this folder will show up inside MATLAB software from now on.
4. Type commands into the MATLAB "Command Window" to work the software. After each MATLAB command, the "return" or "enter" key must be depressed.
5. Go through the steps in tutorial and enter them into MATLAB software to see if it works.
6. MATLAB is case sensitive so "a" and "A" are two different things.
7. Comment statements are preceded by a "%".
8. On-line help for MATLAB can be reached by typing **help** for the full menu or typing help followed by a particular function name or M-file name. For example, **help cos** gives help on the cosine function.
9. The number of digits displayed is not related to the accuracy. To change the format of the display, type **format short e** for scientific notation with 5 decimal places, **format long e**

for scientific notation with 15 significant decimal places and **format bank** for placing two significant digits to the right of the decimal.

10. The commands **who** and **whos** give the names of the variables that have been defined in the workspace.
11. MATLAB codes for almost all examples in each chapter and some figures are available at http://users.ece.gatech.edu/~bonnie/book3/M-files.html.

A. Definition of Variables

1. Variables are assigned numerical values by typing the expression directly, for example, typing
   a = 1+2
   yields: a = 3 (try it!)

   The answer will not be displayed when a semicolon is put at the end of an expression, for example type
   a = 1+2; (try it!)

2. MATLAB utilizes the following arithmetic operators:
   - **+** addition
   - **-** subtraction
   - ***** multiplication
   - / division
   - ^ power operator
   - ' transpose

   A variable can be assigned using a formula that utilizes these operators and either numbers or previously defined variables. For example, since ***a*** was defined previously, the following expression is valid
   b = 2*a; (try it!)

3. To determine the value of a previously defined quantity, type the quantity by itself:
   b
   yields**:** b = 6 (try it!)

4. If your expression does not fit on one line, use an ellipsis (three or more periods at the end of the line) and continue on the next line.
   c = 1+2+3+...
   5+6+7;

5. There are several predefined variables which can be used at any time, in the same manner as user-defined variables:
   i ←→ sqrt(-1)
   j ←→ sqrt(-1)
   pi ←→3.1416...

   For example,
   y= 2*(1+4*j)
   yields: y = 2.0000 + 8.0000i
6. There are also a number of predefined functions that can be used when defining a variable. Some common functions that are used in this text are:

abs	magnitude of a number (absolute value for real numbers)
angle	angle of a complex number, in radians
cos	cosine function, assumes argument is in radians
sin	sine function, assumes argument is in radians
exp	exponential function

   For example, with y defined as above,
   c = abs(y)
   yields: c = 8.2462
   c = angle(y)
   yields: c = 1.3258

   With a=3 as defined previously,
   c = cos(a)
   yields: c = -0.9900
   c = exp(a)
   yields: c = 20.0855

B. Definition of Matrices

1. MATLAB is based on matrix and vector algebra; even scalars are treated as 1x1 matrices. Therefore, vector and matrix operations are as simple as common calculator operations.

2. Vectors can be defined in two ways. The first method is used for arbitrary elements:
   v = [1 3 5 7];
   creates a 1x4 (1 row 4 columns) vector with elements 1, 3, 5 and 7. Commas could be used in place of spaces to separate the elements.

3. Additional elements can be added to the vector:
   v(5) = 8;
   yields the vector v = [1 3 5 7 8].

Previously defined vectors can be used to define a new vector. For example, with **v** defined above

a = [9 10];
b = [v a];

creates the vector b = [1 3 5 7 8 9 10]

4. The second method is used for creating vectors with equally spaced elements:
   t = 0:10:100;
   creates a 1x11 vector t with the elements 0, 10, 20, 30,...,100. Note that the middle number defines the increment.

   If only two numbers are given, then the increment is set to a default of 1:
   k = 0:10;
   creates a 1x11 vector k with the elements 0, 1, 2, ..., 10.

   t= 0:.1:10;
   creates a 1x101 vector t with the elements 0, .1, .2, .3,...,10.

5. Matrices are defined by entering the elements row by row:
   M = [1 2 4; 3 6 8];
   creates the matrix

   $$M = \begin{bmatrix} 1 & 2 & 4 \\ 3 & 6 & 8 \end{bmatrix}$$

   There are a number of special matrices that can be defined:

null matrix:	**M = [ ];**
nxm matrix of zeros:	**M = zeros(n,m);**
nxm matrix of ones:	**M = ones(n,m);**
nxn identity matrix:	**M = eye(n);**

   (n→row, m→column)

6. A particular element of a matrix can be assigned:
   M(1,2) = 5;
   places the number 5 in the first row, second column.
   In this text, matrices are used only in Chapter 12; however, vectors are used throughout the text.

7. Operations and functions that were defined for scalars in the previous section can also be used on vectors and matrices. For example,
   a = [1 2 3];
   b = [4 5 6];
   c = a + b
   yields:
   c = 5 7 9

8. Functions are applied element by element. For example,
   t = 0:10;
   x = cos(2*t);
   yields a vector x with elements equal to cos(2t) for t = 0, 1, 2, ..., 10.

9. Operations that need to be performed element-by-element can be accomplished by preceding the operation by a "**.**" ; For example, to obtain a vector **x** that contains the elements of x(t) = tcos(t) at specific points in time, you cannot simply multiply the vector t with the vector cos(t). Instead you multiply their elements together:
   t = 0:10;
   x = t.*cos(t);
   Note the **.** after t

10. The command **length(x)** returns the length of a vector x and **size(x)** returns the dimension of the matrix x.

C. M-Files

M-files are macros of MATLAB commands that are stored as ordinary text files with the extension "m", that is *filename*.m. Always start M-file name with an alphabetic character. An M-file can be either a function with input and output variables or it can be a list of commands. All of the MATLAB examples in this textbook are contained in M-files that are available from the textbook website. The following describes the use of M-files on a PC version of MATLAB. MATLAB requires that the M-file must be stored in a directory that is specified in the MATLAB path list. For example, consider using MATLAB on a PC with a user-defined M-file stored in a directory called "\MATLAB\MFILES". Then to access that M-file, add that directory to the path by clicking “File” then clicking “Set Path” and following directions.. The M-files associated with this textbook can be downloaded from http://users.ece.gatech.edu/~bonnie/book3/M-files.html.

MATLAB codes for almost all examples in each chapter and some important plots are available at this site. Download them to a user-specified directory. That directory should be added to the path. The M-files that come with MATLAB are already in the path. To create an M-file, click on “File” then click on “New” then click on “M-file” and an editor screen will appear. Type in your MATLAB commands and then save the file to a directory that is in the path. The M-file is run by typing the name of the M-file from the MATLAB command window (without the .m extension). For example, suppose an M-file named **example.m** is located in the path. Then typing **example** from the command prompt runs that M-file.

1. As example of an M-file that defines a function, create a file in your working directory named **func1.m** that contains the following commands:
   function z = somename(y,x)
   z = y + x;

   The following commands typed from within MATLAB demonstrate how this M-file is used:

x = 2;
y = 3;
sumxy = func1(y,x)

2. MATLAB M-files are most efficient when written in a way that utilizes matrix or vector operations. Loops and if statements are available, but should be used sparingly since they are computationally inefficient. An example of the use of the command **for** is
   for k=1:10;
   x(k) = cos(k);
   end ;
   This creates a 1x10 vector **x** containing the cosine of the positive integers from 1 to 10.

3. This operation is performed more efficiently with the commands
   k = 1:10;
   x = cos(k);
   which utilizes a function of a vector instead of a for loop.

4. An if statement can be used to define conditional statements. An example is
   if(a <= 2),
   b = 1;
   elseif(a >=4)
   b = 2;
   else
   b = 3;
   end
   The allowable comparisons between expressions are **>=, <=, <, >, ==,** and **~=.**

Several of the M-files written for this textbook employ a user-defined variable which is defined with the command input. For example, suppose that you want to run an M-file with different values of a variable T. The following command line within the M-file defines the value:
T = input('Input the value of T: ')

Whatever comment is between the quotation marks is displayed to the screen when the M-file is running, and the user must enter an appropriate value.

D. Plotting

Commands covered:

plot
xlabel
ylabel
title
grid
axis
stem
subplot

The command most often used for plotting graphs is *plot*, which creates linear plots of vectors and matrices; **plot(t,y)** plots the vector t on the x-axis versus vector y on the y-axis. There are options on the line type and the color of the plot which are obtained using *plot(t,y,'option').* The linetype options are '-' solid line (default), '--' dashed line, '-.' dot dash line, ':' dotted line. The points in y can be left unconnected and delineated by a variety of symbols: + . * o x. The following colors are available options:

r	red
b	blue
g	green
w	white
k	black

For example, **plot(t,y,'--')** uses a dashed line, **plot(t,y,'*')** uses * at all the points defined in t and y without connecting the points, and **plot(t,y,'g')** uses a solid green line. The options can also be used together, for example, **plot(t,y,'g:')** plots a dotted green line.

To plot two or more graphs on the same set of axes, use the command **plot(t1,y1,t2,y2)**, which plots y1 versus t1 and y2 versus t2.

To label your axes and give the plot a title, type

```
xlabel('time (sec)')
ylabel('step response')
title('My Plot')
```

Finally, add a grid to your plot to make it easier to read. Type **grid**

The problem that you will encounter most often when plotting functions is that MATLAB will scale the axes in a way that is different than you want them to appear. You can easily override the autoscaling of the axes by using the axis command after the plotting command:

axis([xmin xmax ymin ymax]);
where xmin, xmax, ymin, and ymax are numbers corresponding to the limits you desire for the axes. To return to the automatic scaling, simply type **axis**.

To plot more than one graph on the screen, use the command *subplot(mnp)* which partitions the screen into an (m-rows x n-columns) grid where p determines the position of the particular graph counting the upper left corner as p=1. For example,

subplot(211), semilogx(w,magdb); subplot(212), semilogx(w,phase);

plots the bode plot with the log-magnitude plot on top and the phase plot below. Titles and labels can be inserted immediately after the appropriate semilogx command or plot command. To return to a full screen plot, type **subplot(111).**

Lab Assignments:

Let x = [3 2 6 8]' and y = [4 1 3 5]' (x and y should be column vectors).

a. Add the sum of the elements in x to y
b. Raise each element of x to the power specified by the corresponding element in y.
c. Divide each element of y by the corresponding element in x
d. Multiply each element in x by the corresponding element in y, calling the result "z".
e. Add up the elements in z and assign the result to a variable called "w".

Homework:

1. Create a vector x with the elements,

$$x_n = (-1)^{n+1}/(2n-1)$$

Add up the elements of the version of this vector that has 100 elements

2. Given a vector, t, of length n, write down the MATLAB expressions that will correctly compute the following:
   a. $\ln(2 + t + t^2)$
   b. $\cos^2(t) + \sin^2(t)$
   c. $\tan^{-1}(1)$ (this is the inverse tangent function)
   d. $\sec^2(t) + \cot(t) - 1$

   Test that your solution works for t = 1:0.2:2

3. Plot the functions x, x^3, and e^x over the interval $0 < x < 4$